

Flowcharts — ASCII only

Each block = one topic file · pattern triggers · canonical problems · complexity

Master — Problem → Pattern (60-second)

Read constraints + examples

- Contiguous subarray / substring + constraint?
 - All positive / monotone window? → Sliding Window (02-algorithms/sliding-window.md)
 - Negatives or exact sum count? → Prefix Sum + HashMap (01-data-structures/array.md)
- Sorted input OR "minimize max / maximize min"?
 - Find element / boundary → Binary Search (02-algorithms/binary-search.md)
 - Pair / triplet after sort → Two Pointers (02-algorithms/two-pointers.md)
- Tree or graph structure?
 - Shortest path (unweighted) → BFS (01-data-structures/graphs.md)
 - Weighted, non-negative → Dijkstra (02-algorithms/graph.md)
 - Dependencies / ordering → Topo sort (02-algorithms/graph.md)
 - Connected components / cycles → DFS or Union-Find
- "All combinations / permutations"? → Backtracking (02-algorithms/backtracking.md)
- Optimal + overlapping subproblems? → DP (02-algorithms/dynamic-programming.md)
- Greedy local choice (prove exchange)? → Greedy (02-algorithms/greedy.md)
- Top K / merge K sorted / median stream? → Heap (01-data-structures/heap.md)
- Next greater / histogram / brackets? → Monotonic Stack (01-data-structures/stack.md)
- Prefix on strings / dictionary? → Trie (01-data-structures/trie.md)
- Still stuck? → 01-data-structures/ds_tree.md OR 02-algorithms/algorithm_tree.md

00 — Start Here

00-start-here/README.md

- Entry point (topic-first; don't read linearly)
 - Have 4–6 weeks? → Follow week table in 00-start-here/README.md
 - DS: 01-data-structures/README.md · Algo: 02-algorithms/README.md
 - Have 14 days? → Must-nail: Arrays · Hashing · Trees · Graphs · DP · Binary Search
 - + 02-algorithms/problem-deep-dives.md#l4-must-nail-problems
 - + 03-patterns/GOOGLE_INTERVIEW_REVISION.md (Part D)
 - Have 7 days? → GOOGLE_INTERVIEW_REVISION Part D + 2 timed mocks + 3 STAR stories
 - Have 48 hrs? → Hour 0–8: patterns-master + GOOGLE_QUICK_SHEET
 - Hour 8–16: 05-revision/README.md + Quick Revision Triggers per weak topic
 - Hour 16–24: 2 timed mocks + problem-deep-dives L4 list redo
 - Hour 24–48: Sleep → light review only → no new topics
 - AI/ML round? → Not covered in depth here; prep resume + ML fundamentals separately
 - Interview day → 03-patterns/interview-cheatsheet.md (10 min)
 - 05-revision/README.md (complexity + Python gotchas)
 - 04-behavioral/BEHAVIORAL_GOOGLELINESS.md (STAR titles cold)

01 — Data Structures

01-data-structures/README.md

- DS hub – study order · L4 priorities · how to read each file
 - Pick path: first learn | revision | "which DS?" → ds_tree.md
 - Must-nail L4: Arrays · Hashing · Trees · Graphs · Strings · Stacks · Heaps

01-data-structures/array.md

- Arrays [O(1) access · O(n) insert/delete]
 - Two Pointers → sorted array / linked list → target sum · remove duplicates · container with most water
 - Sliding Window → subarray/substring → max sum subarray · longest substring without repeat
 - Prefix Sum → range sum queries → subarray sum equals K · product except self
 - Kadane's → max subarray → O(n) time O(1) space
 - Difference Array → range updates → O(1) update, O(n) query
 - Boyer-Moore Majority Vote → majority element ($> N/2$) → O(n) time O(1) space
 - Floyd's Cycle Detection on Arrays → duplicate number in [1, N] → O(n) time O(1) space
 - Sorting-based → merge intervals · meeting rooms · 3Sum

01-data-structures/hashtable.md

- Hash Maps [O(1) avg insert/search/delete]

- Frequency Map → anagram groups · top-K elements · first unique char
- Two Sum pattern → one-pass $O(n)$ · complement lookup
- Grouping → group anagrams · isomorphic strings
- Swap-with-Last → $O(1)$ insert, delete, getRandom (map + dynamic array)
- Rolling Hash → Rabin-Karp substring search · repeated DNA sequences

01-data-structures/string.md

- Strings [immutable char array · $O(k)$ substring · $O(n^2)$ concat in loop]
 - Frequency Map → anagram check · group anagrams · find all anagrams in string
 - Two Pointers → palindrome check · longest palindromic substring (expand around center)
 - Sliding Window (variable) → longest substring without repeat · min window substring · at-most-K distinct
 - Sliding Window (fixed) → find all anagrams · permutation in string
 - Prefix Sum on chars → count substrings with exactly K vowels / specific char counts
 - String Hashing → detect duplicate substrings · repeated DNA sequences

01-data-structures/linked-list.md

- Linked Lists [$O(n)$ access · $O(1)$ insert at pointer]
 - Fast/Slow Pointers → cycle detect · middle of list · cycle entry point
 - Dummy Node → merge sorted lists · remove Nth from end
 - Reverse In-Place → reverse linked list · reverse groups of K
 - Two-list merge → merge K sorted lists (use heap)

01-data-structures/stack.md

- Stacks [$O(1)$ push/pop]
 - Monotonic Stack (decreasing) → Next Greater Element · Daily Temperatures · Largest Rectangle in Histogram
 - Monotonic Stack (increasing) → Next Smaller Element · Trapping Rain Water
 - Bracket Matching → valid parentheses · min removes · score of brackets
 - Expression evaluation → basic calculator · reverse Polish notation

01-data-structures/queue.md

- Queues / Deques [$O(1)$ enqueue/dequeue]
 - BFS → shortest path unweighted · level-order tree · rotting oranges · word ladder
 - Monotonic Deque (max) → sliding window maximum · $O(n)$ time
 - Priority Queue (heap) → Dijkstra · top-K · median maintenance

01-data-structures/tree.md

- Trees / BST [$O(\log n)$ avg BST ops]
 - DFS Pre-order → serialize/deserialize · construct from preorder
 - DFS In-order → BST sorted output · kth smallest · validate BST
 - DFS Post-order → subtree problems · diameter · balanced check
 - BFS (level) → level-order · right side view · zigzag traversal
 - LCA → binary lifting / recursive $O(n)$
 - Path Sum → root-to-leaf · any path · max path sum
 - Morris Traversal → $O(1)$ space inorder

01-data-structures/graphs.md

- Graphs [V vertices · E edges]
 - BFS → $O(V+E)$ · shortest path (unweighted) · bipartite check · walls and gates
 - DFS → $O(V+E)$ · cycle detection · connected components · number of islands
 - Topological Sort (Kahn's) → $O(V+E)$ · course schedule · build order
 - Union-Find → $O(\alpha \cdot n) \approx O(1)$ · dynamic connectivity · redundant connection
 - Dijkstra (min-heap) → $O((V+E)\log V)$ · network delay · cheapest flights
 - Bellman-Ford → $O(VE)$ · negative weights · detect negative cycle

01-data-structures/heap.md

- Heaps [$O(\log n)$ push/pop · $O(n)$ heapify]
 - Top-K largest → min-heap size K → $O(n \log k)$
 - Top-K smallest → max-heap size K → negate values in Python
 - Merge K sorted lists → min-heap of (val, list_idx, elem_idx)
 - Two-Heap Median → max-heap left | min-heap right → balance on insert
 - Dijkstra → (dist, node) in min-heap → relax neighbors

01-data-structures/trie.md

- Tries [$O(m)$ insert/search — m = word length]
 - Prefix search → autocomplete · starts-with check · implement Trie
 - Word Break → DP + Trie to find valid splits
 - Word Search II → Trie + DFS backtrack on grid
 - XOR Maximum → binary Trie for max XOR of two numbers

01-data-structures/segment-tree.md

- Segment Trees [$O(\log n)$ query/update · $O(n)$ build]
 - Range Sum Query → build once · point update · range query
 - Range Min/Max Query → same structure · different merge function

└─ Lazy Propagation → range update in $O(\log n)$ → range sum with add

01-data-structures/advanced-structures.md

- └─ Advanced [Know use-case; unlikely to implement from scratch at L4]
 - └─ LRU / LFU Cache → HashMap + doubly linked list · $O(1)$ get/put
 - └─ Fenwick Tree (BIT) → prefix sums with $O(\log n)$ update · simpler than seg tree
 - └─ Bloom Filter → probabilistic set membership · false positives OK · no false negatives
 - └─ Skip List → sorted structure with $O(\log n)$ avg · alternative to balanced BST

01-data-structures/ds_tree.md

- └─ Data Structure Decision Tree [problem constraints → optimal data structure]
 - └─ $O(1)$ lookup by key (unordered) → HashMap / HashSet
 - └─ $O(\log n)$ ordered ops (floor/ceil) → TreeMap / BST
 - └─ $O(1)$ min or max peek → Heap (min or max)
 - └─ Running median → Two heaps (max + min)
 - └─ $O(1)$ both-end access/removal → Deque
 - └─ LIFO / undo / NGE → Stack (Monotonic Stack)
 - └─ Prefix word / autocomplete → Trie
 - └─ Static range sum/min/max → Prefix sum array
 - └─ Dynamic range sum/min/max → Segment Tree / BIT
 - └─ Dynamic connectivity → Union-Find

02 — Algorithms (Flat Files)

02-algorithms/two-pointers.md

- └─ Two Pointers [Converging vs same-direction vs multi-sequence scan]
 - └─ Converging (opposite ends) → sorted array sum · palindrome check · container with most water
 - └─ Fast/Slow (Floyd's) → cycle detection · cycle entry point · middle of linked list
 - └─ Same-direction (read/write) → in-place partition · move zeros · remove duplicates
 - └─ Dutch National Flag → 3-way partition (lo, mid, hi) for 3-category sorting in $O(n)$
 - └─ Two-array merge → sorted search space merge · merge K sorted (pair-wise or heap)

02-algorithms/sliding-window.md

- └─ Sliding Window [running aggregates over contiguous sub-ranges]
 - └─ Fixed size → build initial K · slide by adding right & deleting left-K
 - └─ Variable size (longest/shortest) → expand right always · shrink left while constraint violated
 - └─ Frequency map/match-count → track character constraints (Anagrams, Min Window Substring)
 - └─ Monotonic deque → sliding window maximum/minimum in $O(n)$ total time
 - └─ At-Most Trick → solve "Exactly K" via $\text{atMost}(K) - \text{atMost}(K-1)$ difference

02-algorithms/binary-search.md

- └─ Binary Search
 - └─ Standard → $O(\log n)$ · sorted array · search insert position
 - └─ Lower bound → first position $\geq$ target → `bisect_left`
 - └─ Upper bound → first position $>$ target → `bisect_right`
 - └─ Rotated array → find pivot first · then binary search half
 - └─ BS on answer → minimize max / maximize min → guess + validate in $O(n)$
 - └─ 2D matrix → treat as 1D with `idx // cols`, `idx % cols`

02-algorithms/sorting.md

- └─ Sorting [Ordering invariants & selection tradeoffs]
 - └─ Merge Sort → $O(n \log n)$ stable · count inversions
 - └─ Quick Sort → $O(n \log n)$ avg $O(n^2)$ worst · partition around randomized pivot
 - └─ Heap Sort → $O(n \log n)$ · $O(1)$ space · not stable
 - └─ Counting & Radix Sort → $O(n+k)$ / $O(nd)$ · bounded integer ranges
 - └─ Dutch National Flag → 3-way partition for 0s, 1s, 2s → $O(n)$ time $O(1)$ space
 - └─ QuickSelect → $O(n)$ average time for Kth order statistic
 - └─ External Merge Sort → split runs + K-way merge for data > RAM
 - └─ Custom sort → sort by multiple keys · Schwartzian transform in Python

02-algorithms/greedy.md

- └─ Greedy [Irrevocable local choices for global optimum]
 - └─ Interval scheduling → sort by end time · earliest finish selection
 - └─ Interval merging → sort by start time · overlap merging
 - └─ Jump Game → track farthest reachable index · $O(n)$
 - └─ Candy Distribution → two-pass constraints (L→R, R→L) · merge via max
 - └─ Partition Labels → last occurrence map · extend boundary greedily
 - └─ Priority Queue Greedy / IPO → sort by constraint + max-heap of profits
 - └─ Huffman & Task Scheduler → frequency-based merge/slots counting

02-algorithms/divide-and-conquer.md

- └─ Divide & Conquer
 - └─ Merge Sort → inversion count as side-effect

- └ Quick Select → $O(n)$ avg k-th largest without full sort
- └ Closest Pair of Points → $O(n \log n)$ · split + merge strip

02-algorithms/backtracking.md

- └ Backtracking [$O(2^n)$ subsets · $O(n!)$ permutations]
 - └ Subsets → choose/skip at each index · power set
 - └ Permutations → swap in-place or used[] array
 - └ Combinations → start index to avoid duplicates · k-of-n
 - └ N-Queens → row-by-row · col+diag sets for $O(1)$ conflict check
 - └ Sudoku Solver → try 1-9 · recurse · undo on failure

02-algorithms/dynamic-programming.md

- └ DP complete guide → see § 02 – Dynamic Programming below
 - └ 15 patterns · universal 4-step recipe · 12 bugs · L4 must-nail

02-algorithms/recursion.md

- └ Recursion complete guide → see § 02 – Recursion below
 - └ 8 patterns · Aditya Verma 4-step · 15 bugs · combination family · LC table

02-algorithms/recursion-to-dp.md

- └ Memo → tabulate conversion (companion to recursion.md)

02-algorithms/union-find.md

- └ Union-Find [$O(\alpha \cdot n) \approx O(1)$ with path compress + rank]
 - └ Connected components → find() with path compression
 - └ Cycle detection in undirected graph → union returns False if same component
 - └ Kruskal's MST → sort edges by weight · union if different components
 - └ Dynamic connectivity → accounts merge · friend circles

02-algorithms/bit-manipulation.md

- └ Bit Manipulation [Direct integer register operations]
 - └ XOR Tricks → single/missing number · find two non-repeating (partition lowest set bit)
 - └ Bit Masking → subset enumeration [$0..2^N-1$] · bitmask DP [$N \leq 20$ states]
 - └ Submask Enumeration → sub = (sub-1) & mask loop → $O(3^N)$ total complexity
 - └ Bitmask + BFS → state space search via (node, visited_mask) to allow revisits
 - └ Power of 2 check → $n > 0$ and $(n \& (n-1)) == 0$
 - └ Brian Kernighan → count set bits via $n \&= (n-1)$ until 0
 - └ XOR Trie → binary prefix tree for $O(32)$ maximum XOR queries

02-algorithms/graph.md

- └ Graph Algorithms
 - └ BFS → deque · visited set · layer-by-layer
 - └ DFS → recursive or explicit stack · pre/post hooks
 - └ Topological Sort (Kahn's) → in-degree array · queue of 0-in-degree nodes
 - └ Dijkstra → min-heap · relax if $\text{dist}[\text{nbr}] > \text{dist}[u] + w$
 - └ Bellman-Ford → $V-1$ rounds of all-edge relaxation · detect neg cycle on round V
 - └ 0-1 BFS → deque [0-cost to front · 1-cost to back] → $O(V+E)$
 - └ Minimum Spanning Tree (MST)
 - └ Kruskal's → sort edges + DSU → $O(E \log E)$
 - └ Prim's → min-heap growing frontier → $O(E \log V)$

02-algorithms/advanced-graphs.md

- └ Advanced Graphs
 - └ Floyd-Warshall → $O(V^3)$ all-pairs · $\text{dp}[i][j] = \min(\text{dp}[i][j], \text{dp}[i][k] + \text{dp}[k][j])$
 - └ Prim's MST → min-heap of (weight, node) · visited set
 - └ Bridges & Articulation Points → Tarjan's DFS · low[] and disc[] arrays
 - └ A* → heuristic-guided BFS · $f = g + h$

02-algorithms/string.md

- └ String Algorithms
 - └ KMP → $O(n+m)$ · failure function (LPS array) · no backtrack on text
 - └ Rabin-Karp → $O(n+m)$ avg · rolling hash · substring search
 - └ Z-algorithm → $O(n)$ · $Z[i]$ = length of longest prefix match at i
 - └ Manacher's → $O(n)$ longest palindromic substring

02-algorithms/maths.md

- └ Math [Number theory & algebraic invariants]
 - └ GCD / LCM → Euclidean: $\text{gcd}(a,b) = \text{gcd}(b, a\%b)$ · $\text{lcm} = (a // \text{gcd}) * b$ (no overflow)
 - └ Extended GCD & Modular Inverse → $ax + by = \text{gcd}(a,b)$ · inverse via Fermat's (M prime) or Ext-GCD
 - └ Sieve of Eratosthenes → $O(n \log \log n)$ prime marking · SPF sieve for $O(\log k)$ factorization
 - └ Binary Exponentiation → repeated squaring for $x^n \bmod M$ in $O(\log n)$ · matrix exponentiation
 - └ Modular Arithmetic → apply mod at each addition/multiplication · $(a-b+\text{MOD})\% \text{MOD}$ for subtraction
 - └ Combinatorics → $nCr \bmod p$ via precomputed fact[] and inv_fact[] arrays in $O(1)$ query
 - └ Integer Geometry → collinearity via cross-product · slope tuples (dy//g, dx//g) to avoid floats

02-algorithms/searching.md

- └ Searching [$O(\log n)$ binary search vs $O(n)$ linear scan]
 - └ Template 1 (Exact Match) → $lo \leq hi$ · return mid on match
 - └ Template 2 (Boundary Search) → $lo < hi$ · find first True of monotone predicate
 - └ Binary Search on Answer → minimize max / maximize min → range guess + $O(n)$ validate
 - └ Rotated Sorted Array → identify sorted half first → search target or find min
 - └ Peak Element → compare mid with mid+1 → climb gradient to local maximum
 - └ Median of Two Sorted Arrays → partition both arrays → $O(\log(\min(M, N)))$

02-algorithms/concurrency.md

- └ Concurrency [SDE-3 depth; SDE-2: know the patterns]
 - └ Visibility vs Atomicity → volatile fixes visibility; AtomicInteger fixes atomicity
 - └ Deadlock (4 conditions) → Mutual Exclusion · Hold+Wait · No Preemption · Circular Wait
 - └ Fix → break any one; most common: resource ordering (break circular wait)
 - └ Bounded Blocking Queue → Condition + deque; enqueue waits when full, dequeue waits when empty
 - └ Dining Philosophers → circular deadlock example; fix: philosopher N reverses fork pick-up order
 - └ Semaphore vs Mutex → semaphore counts (N permits); mutex is binary ownership lock

02-algorithms/system-design-algorithms.md

- └ System Design Algorithms [distributed systems building blocks]
 - └ Bloom Filter → probabilistic set membership · false positives OK · no false negatives
 - └ Use case: check username taken without DB hit · malicious URL filter · Cassandra/BigTable
 - └ HyperLogLog → estimate unique elements · 1.5 KB for 1B users · 2% error
 - └ Use case: DAU counting · cardinality estimation in analytics pipelines
 - └ Consistent Hashing → map nodes+keys to circular ring · only $1/N$ keys reshuffle on node change
 - └ Use case: distributed cache · Dynamo/Cassandra · add virtual nodes for uniform distribution
 - └ Rate Limiting → token bucket (smooth bursts) vs leaky bucket (strict rate) vs sliding window log
 - └ Count-Min Sketch → frequency estimation in stream · $O(1)$ update · sublinear space

02-algorithms/miscellaneous.md

- └ Miscellaneous Advanced Structures [SDE-3 level; SDE-2: know use-cases]
 - └ Fenwick Tree (BIT) → dynamic prefix sums · $O(\log n)$ update+query · 1-indexed · ~5x less code than seg tree
 - └ Use case: count inversions · range sum with point updates · coordinate compression problems
 - └ Sparse Table → $O(n \log n)$ build · $O(1)$ range min/max · static only (no updates) · idempotent ops
 - └ Sweep Line → sort events by x · process with active set · $O(n \log n)$ total
 - └ Use case: skyline problem · rectangle area union · interval overlap counting
 - └ LRU Cache → HashMap + doubly linked list · $O(1)$ get and put · move to head on access
 - └ LFU Cache → 3 maps (key→val, key→freq, freq→OrderedDict) + min_freq tracker · $O(1)$ all ops

02-algorithms/algorithm_tree.md

- └ Algorithm Decision Tree [problem type → algorithm → template]
 - └ Sorting & Partitioning → Merge/Quick/QuickSelect/Dutch Flag/Counting/Custom comparator
 - └ Searching → Binary Search variants · BS on answer · Rotated array · Median finding
 - └ Graph traversal → BFS (shortest) · DFS (paths/cycles) · Topo sort (dependencies)
 - └ Optimization → Greedy (local optimal) · DP (overlapping subproblems) · Divide & Conquer
 - └ Design patterns → Monotonic Stack/Deque · Two Pointers · Sliding Window · Union-Find

02 — Dynamic Programming

02-algorithms/dynamic-programming.md

- └ Complete DP guide – 15 patterns · 12 bugs · interview bank
 - └ UNIVERSAL RECIPE (always in this order)
 - └ 1. Recursive brute force (correct base cases)
 - └ 2. Memoization (top-down)
 - └ 3. Tabulation (bottom-up)
 - └ 4. Space optimize (rolling row / two vars)
 - └ CHOOSE PATTERN IN 30s
 - └ Items once + capacity/sum → 0/1 Knapsack (w loop BACKWARD in 1D)
 - └ Unlimited copies + capacity → Unbounded Knapsack (w loop FORWARD)
 - └ $dp[i]$ from $dp[i-1]$, $dp[i-2]$ → Linear / Fibonacci
 - └ Two strings align/match → LCS family
 - └ Longest increasing chain → LIS ($O(n^2)$ or $O(n \log n)$ patience sort)
 - └ Best contiguous subarray → Kadane
 - └ Split interval $[i, j]$ at k → Interval DP / MCM
 - └ Subtree rob/skip/path at root → Tree DP (postorder, return tuple)
 - └ $N \leq 20$, visit all / assign → Bitmask DP

- Count in [L,R] digit property → Digit DP (tight flag)
- Buy/sell/cooldown/fee/k tx → Stock state machine
- Probability / optimal play → Expected value / minimax DP
- 15 PATTERNS → CANONICAL PROBLEMS
 - 1 Linear/Fibonacci → climb stairs · house robber · decode ways · word break
 - 2 0/1 Knapsack → subset sum · target sum · partition equal subset
 - 3 Unbounded Knapsack → coin change I/II · perfect squares · combo sum IV
 - 4 LCS family → LCS · edit distance · interleaving · distinct subseq
 - 5 LIS → LIS · Russian doll envelopes · longest string chain
 - 6 Kadane → max subarray · max product · circular max
 - 7 Interval DP → burst balloons · palindrome cuts · MCM · strange printer
 - 8 Grid DP → unique paths · min path sum · maximal square · dungeon (reverse fill)
 - 9 Tree DP → house robber III · max path sum · cameras · diameter
 - 10 Bitmask DP → TSP · shortest path all nodes · smallest sufficient team
 - 11 Stock machine → buy/sell I-IV · cooldown · transaction fee · k transactions
 - 12 String/palindrome → LPS · palindrome partitioning · regex matching · wildcard
 - 13 Digit DP → numbers at most N · unique digits · digit sum = K
 - 14 Probability/game → knight probability · new 21 game · stone game · egg drop
 - 15 Advanced (stretch) → deque opt · CHT · SOS DP · Knuth · WQS binary search
- L4 MUST-NAIL
 - house robber · coin change · word break · LIS · LCS · edit distance
 - full pseudocode in problem-deep-dives.md
- TOP BUGS (memorize these)
 - 0/1 knapsack 1D → iterate w BACKWARD; unbounded → FORWARD
 - Count problems → dp[0] = 1 (one empty way)
 - Interval DP → fill by increasing length, not row index
 - Burst balloons → k is LAST to burst, not first
 - Regex * → dp(i,j-2) for zero occurrences
 - Dungeon → fill backwards (forward impossible)
 - Stock cooldown → save prev_sold before updating sold
- WHEN NOT DP → greedy (local optimal) · D&C (no overlap) · BFS/Dijkstra (path) · math formula

02 — Recursion

02-algorithms/recursion.md

- Complete recursion reference – 8 patterns · 15 bugs · tiered question bank
- ADITYA VERMA 4-STEP
 - 1. Draw choice diagram (branches per level)
 - 2. Define IP/OP (what shrinks · what you build)
 - 3. Explore all paths (include/exclude or choose-next)
 - 4. Unchoose / restore state (undo mutations)
- BASE CASE CHECKLIST
 - Empty input (n==0, node==None, i==len(s))
 - Single element / leaf
 - Constraint met (remaining==0, target found)
- RETURN VALUE DESIGN (pick one)
 - Single value → height, count, bool
 - Tuple → (rob, skip) · (is_bst, min, max, size)
 - Global + local → diameter, max path sum
- 8 PATTERNS → SIGNAL → CANONICAL PROBLEMS
 - 1 Include/Exclude → "all subsets" → subsets I/II · target sum
 - 2 Permutations → "all orderings" → permutations I/II · letter case perm
 - 3 IP/OP + guard → "generate valid X" → generate parentheses · phone combos · restore IP
 - 4 Divide & combine → "split · merge" → merge sort · quickselect · unique BSTs II
 - 5 Mathematical → "recurrence formula" → fibonacci · tower of Hanoi · josephus
 - 6 Tree/graph DFS → "traverse / property" → flood fill · islands · topo · validate BST · LCA
 - 7 Constraint satisfy → "place N with rules" → N-Queens · Sudoku · word search II (trie prune)
 - 8 Memo bridge → "TLE + overlap" → regex/wildcard · combo sum IV → DP
- COMBINATION FAMILY (same file)
 - Combo sum I → reuse: backtrack(i, ...) same index

- Combo sum II → once each: backtrack(i+1) + sort/skip dupes (i > start)
 - Combo sum III → k digits 1–9, both len==k AND sum==0
 - Combo sum IV → order matters → really unbounded knapsack DP
- L4 MUST-NAIL
 - subsets · permutations · combination sum · generate parentheses · word search
 - LC table with gotchas in recursion.md § Full Interview Questions
- TOP BUGS
 - Snapshot path → append(current[:]) not current
 - Restore state → path.pop() / unmark visited on return
 - Combo dedup → i > start (not i > 0)
 - Perm dedup → not used[i-1] when nums[i]==nums[i-1]
 - Graph → mark visited BEFORE recurse
 - Directed cycle → 3-color (white/gray/black), not one visited set
 - @lru_cache → args must be hashable (tuple not list)
- WHEN NOT RECURSE → DP tabulation (overlap + optimal only) · iterative stack (depth > 10⁴) · BFS
 - greedy · union-find

03 — Patterns

- 03-patterns/README.md
 - Patterns folder index
- 03-patterns/patterns-master.md
 - Pattern Recognition Master [READ FIRST – before drilling problems]
 - Trigger: sorted + target sum → Two Pointers
 - Trigger: subarray/substring + constraint → Sliding Window
 - Trigger: sorted search + O(log n) → Binary Search
 - Trigger: shortest path → BFS
 - Trigger: all paths / combinations → DFS / Backtrack
 - Trigger: optimal substructure → DP
 - Trigger: K largest/smallest → Heap
- 03-patterns/interview-cheatsheet.md
 - One-page interview cheatsheet [Read on interview day – 10 min max]
 - All patterns + complexity + Python snippet
- 03-patterns/GOOGLE_QUICK_SHEET.md
 - Google-specific [What they reward / red flags]
 - Reward: clean code · optimal complexity · edge cases stated upfront
 - Red flag: brute force without improvement · silent coding · no test cases
- 03-patterns/GOOGLE_INTERVIEW_REVISION.md
 - Deep revision guide [topic-by-topic Google lens]
 - Use in Week 4 and 48-hr sprint
- 03-patterns/TOPIC QUESTIONS LOGIC AND TRICKS.md
 - 100+ canonical problems [logic + trickiness per problem]
 - Use to identify why a problem is tricky, not just what the solution is
- 03-patterns/canonical-questions.md
 - Canonical Questions – Key Insight Index [one-line insight per problem]
 - Arrays: Two Sum → check-before-store · Kadane → reset when prefix negative · Trapping Rain → min(left_max, right_max) - h[i]
 - Strings: Min Window → two counters (need/have) · Longest Palindrome → expand odd+even at each index
 - Trees: LCA any tree → return node when found; LCA = where both sides non-null
 - Graphs: Word Ladder → BFS on graph where edge = 1-char difference
 - DP: Coin Change → unbounded knapsack · Burst Balloons → last balloon in interval = dp[i][j]
 - Use: check if you know the key insight cold before opening solution
- 03-patterns/leetcode-variants.md
 - LC variants by pattern [grouped for drilling]
 - Use after reading patterns-master – drill 2–3 variants per pattern
- 03-patterns/lld.md
 - Low-level design [LRU · parking lot · in-memory FS – SDE-3 / hybrid rounds]
- 03-patterns/system-design.md
 - System design stubs [SDE-2 scope · optional unless on your loop]
 - Pair with 02-algorithms/system-design-algorithms.md for building blocks

04 — Behavioral

04-behavioral/README.md

- └ Google 4 Attributes
 - └ General Cognitive Ability → structured thinking · how you solve unknowns
 - └ Leadership → influence without authority · step up or step back
 - └ Role-related Knowledge → depth in your craft
 - └ Googliness → honest · scrappy · team-first

04-behavioral/behavioral.md

- └ STAR Story Index
 - └ Structure: Situation → Task → Action → Result (quantify)
 - └ Story types: conflict · failure · ambiguity · leadership · cross-team
 - └ Rehearse: 3 stories minimum · titles + outcomes cold

04-behavioral/BEHAVIORAL_GOOGLINESS.md

- └ Four Google Attributes – Deep Dive
 - └ GCA → decompose before diving · handle new info mid-conversation · justify tradeoffs
 - └ Signal: "Before I start coding, let me think about edge cases..."
 - └ Leadership → emergent not positional · influence without authority · own the outcome
 - └ Signal: "No one was owning X, so I set up a sync and drove it to completion..."
 - └ Role-Related Knowledge → know WHY you made decisions · failure modes not just happy paths
 - └ Signal: "We chose eventual consistency because strong consistency added 3x latency..."
 - └ Googliness → scrappy · intellectually humble · credit others · bias for action
 - └ Signal: "I didn't know X, so I read the source code and ran an experiment..."
 - └ Red Flags → sole hero stories · no quantified results · always "we" never "I" · badmouthing
 - └ Pre-interview checklist → 7 story types ready: impact · conflict · failure · ambiguity · tech deep-dive · influence · fast learning

04-behavioral/googliness-round.md

- └ Googliness Round [Proving team effectiveness & structured behavior]
 - └ Core Evaluation → GCA · Leadership · Googliness · Role-Related Knowledge
 - └ Answer Framework (STAR+R) → Situation (context) · Task (responsibility) · Action (decisions) · Result (quantified) · Reflection (lessons)
 - └ 5 Key Signals → Ownership (close gaps) · Humility (admit mistakes) · Collaboration (team value) · Judgment (tradeoffs) · Impact (metrics)
 - └ 7 Story Bank Must-Haves → Big impact · Disagreement · Failure · Ambiguity · Influence without authority · Learning fast · Helping others
 - └ Red Flags → too much "we" · no metrics · blaming others · fake failures

05 — Revision

05-revision/README.md

- └ Complexity Quick-Reference
 - └ Sorting → Merge $O(n \log n)$ stable · Quick $O(n \log n)$ avg · Heap $O(n \log n)$ $O(1)$ space
 - └ DS → Array $O(1)$ access · HashMap $O(1)$ avg · BST $O(\log n)$ avg · Heap $O(\log n)$ push
 - └ Graphs → BFS/DFS $O(V+E)$ · Dijkstra $O((V+E)\log V)$ · Floyd $O(V^3)$ · Union-Find $O(\alpha \cdot n)$
 - └ Techniques → Binary Search $O(\log n)$ · DP $O(\text{states} \times \text{transition})$ · Backtrack $O(2^n \text{ or } n!)$
- └ Python Gotchas
 - └ $-7 // 2 == -4$ (floors toward $-\infty$) → use $-((-a)//b)$ for ceiling
 - └ `list.pop(0)` is $O(n)$ → use `deque.popleft()` for queues
 - └ `heapq` is min-heap → negate for max-heap
 - └ `dict.get(k, 0)` → no `KeyError` on missing key
 - └ `arr[:]` → shallow copy (`arr = arr2` is a reference, not copy)
 - └ `"".join(chars)` → $O(n)$ string build (`s += c` is $O(n^2)$)

05-revision/mock-log.md

- └ Log every mock – problem · pattern · failure mode · redo date

05-revision/coding-rubric.md

- └ Google Coding Rubric
 - └ Problem understanding → restate + clarify constraints + examples
 - └ Approach → brute → optimal → justify complexity
 - └ Code quality → clean · named vars · no magic numbers
 - └ Testing → trace examples · edge cases (empty · single · overflow)
 - └ Communication → narrate decisions · don't go silent

Practice loop (what to do with this file)

Weekly cycle

- └ Learn → 01/02 topic file (Core Algorithms section) + patterns-master triggers
- └ Drill → 3–5 problems on judge; use problem-deep-dives after attempt
- └ Mock → 45–60 min timed · log in 05-revision/mock-log.md
- └ Redo → failures from mock log · cold at +2 days and +7 days
- └ Revise → FLOWCHARTS.md (this file) OR Quick Revision Triggers in topic files

Readiness signal (DSA)

- └ Can name pattern in ~60s for L4 must-nail list
- └ Medium problems cold in ~25–35 min with communication
- └ 4+ mocks logged with improving failure modes